FastBPE: Benchmarking and Optimizing Tokenizer Spee

Research-style benchmark report

FastBPE benchmarks tokenizer throughput, latency, memory, batching behavior, and domain sensitivity across English, code, web, and technical text. In the current five-trial run on Windows-10-10.0.26200-SP0, tiktoken was the strongest average MB/s performer at 4.780 MB/s. SentencePiece remained competitive and led the code domain at 5.035 MB/s. The cached Python tokenizer improved on the naive Python baseline by 1.18x in MB/s with higher memory usage.

Hypotheses

H1. Production native tokenizer libraries will outperform readable Python baselines on throughput and latency.

H2. Tokenizer rankings will change across English prose, code, noisy web text, and technical markdown-like text.

H3. Repeated-substring caching will improve the custom Python baseline while increasing memory usage.

H4. Batch encoding will favor tokenizers that expose efficient multi-document paths, but speed gains will not be uniform.

H5. Exact token-ID compatibility cannot be claimed unless the tokenizer intentionally matches the reference vocabulary and merge logic.

Methodology and Experimental Design

Environment. Platform: Windows-10-10.0.26200-SP0. Python: 3.11.9. Processor: Intel64 Family 6 Model 189 Stepping 1, GenuineIntel.

Tokenizers. tiktoken, tiktoken_cached, hf, sentencepiece, naive, cached.

Domains. code, english, technical, web.

Controls. Warmup before timing, separate cold-start measurement, identical document sets per run, five trials, and raw CSV/JSON output.

Datasets. This run used persisted synthetic corpora in data/ rather than the in-memory homogeneous fallback. The corpus was diversified specifically to reduce misleading cache behavior from repeated identical documents.

Exactness. tiktoken is the reference tokenizer, but this run did not include a non-reference tokenizer that claimed token-ID compatibility, so exact-match tables are absent by design.

Experiments

Experiment 1: Overall tokenizer speed, measured in MB/s, tokens/s, documents/s, and latency percentiles.

Experiment 2: Domain sensitivity across clean English, code, noisy web text, and technical text.

Experiment 3: Input-length scaling across the available length buckets in the current corpus.

Experiment 4: Batch versus single encoding, reported for tokenizers that support encode_batch.

Experiment 5: Exactness testing, included as a framework feature but not exercised for a non-reference compatible tokenizer in this run.

Experiment 6: Caching optimization, comparing the naive and cached Python prototypes.

Overall Results by Tokenizer

This table aggregates mean performance across all domains and trials.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	4.780	1126952	0.2195	34723.1
sentencepiece	2.128	1078825	0.5194	59718.0
tiktoken_cached	1.855	437717	0.5646	243773.2
hf	1.621	473489	0.6517	36223.0
cached	1.349	1217571	0.8302	400323.9
naive	1.145	1033837	0.9336	60392.2

Top Tokenizer-Domain Configurations

These are the strongest individual tokenizer-domain combinations by MB/s.

tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	technical	5.715	1153084	0.2563	20807.0
tiktoken	code	5.035	1174886	0.4045	64324.0
tiktoken	english	4.479	1101243	0.1689	31258.2
tiktoken	web	3.892	1078596	0.0482	22503.2
tiktoken_cached	code	2.782	649223	0.7400	276448.6
sentencepiece	code	2.343	982552	0.8812	115848.0
sentencepiece	english	2.273	1099567	0.3384	38096.0
tiktoken_cached	english	2.018	496108	0.3863	241707.0
sentencepiece	technical	1.973	1028027	0.7608	39344.0
hf	code	1.969	490389	1.0373	61696.0

Domain Sensitivity: MB/s by Domain

Throughput shifts materially by domain, which changes the ranking of libraries.

domain	cached	hf	naive	sentencepiece	tiktoken	tiktoken_cached
code	1.765	1.969	1.194	2.343	5.035	2.782343231865716
english	1.462	1.660	1.144	2.273	4.479	2.017852954859546
technical	1.022	1.576	1.247	1.973	5.715	1.5976657298240595
web	1.147	1.281	0.995	1.924	3.892	1.0218499200704934

Cold Start and Batch Results

Cold start is reported for all tokenizers. Batch metrics are available only for adapters with batch support.

tokenizer	metric	value	docs_per_s	tokens_per_s	batch_latency_ms
tiktoken	cold_start_ms	0.1142			
naive	cold_start_ms	0.1707			
cached	cold_start_ms	0.1846			
tiktoken_cached	cold_start_ms	0.2194			
sentencepiece	cold_start_ms	0.5428			
hf	cold_start_ms	0.6362			
cached			13481.1	6889580	1.2202
naive			12363.8	6057127	1.2972
tiktoken_cached			9340.1	1577443	1.2675
hf			6433.9	1301140	1.9184
sentencepiece			5050.5	2138750	1.7950

Input-Length Scaling Summary

The current persisted corpus produced two dominant length buckets, so the length-scaling analysis is informative but not yet broad.

tokenizer	length_bucket	latency_ms	bytes	tokens
cached	1025-4096	1.2971	1826.3	1581.7
cached	257-1024	0.4074	579.5	477.6
cached	4097+	2.7235	7772.0	6960.0
cached	65-256	0.1261	134.7	123.5
hf	1025-4096	1.0012	1826.3	450.3
hf	257-1024	0.3367	579.5	169.3
hf	4097+	3.5730	7772.0	2082.0
hf	65-256	0.1103	134.7	55.1
naive	1025-4096	1.4421	1826.3	1581.7
naive	257-1024	0.4888	579.5	477.6
naive	4097+	4.7670	7772.0	6960.0
naive	65-256	0.1367	134.7	123.5
sentencepiece	1025-4096	0.8094	1826.3	808.4
sentencepiece	257-1024	0.2536	579.5	272.3
sentencepiece	4097+	3.3427	7772.0	3262.0
sentencepiece	65-256	0.0713	134.7	87.0
tiktoken	1025-4096	0.3319	1826.3	387.3
tiktoken	257-1024	0.1239	579.5	134.9
tiktoken	4097+	1.5242	7772.0	1693.0
tiktoken	65-256	0.0368	134.7	38.6
tiktoken_cached	1025-4096	0.8525	1826.3	387.3
tiktoken_cached	257-1024	0.2937	579.5	134.9
tiktoken_cached	4097+	1.9050	7772.0	1693.0
tiktoken_cached	65-256	0.1389	134.7	38.6

Caching Optimization Results

Repeated-substring caching improved throughput and latency relative to the naive Python baseline, with a memory cost.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_byte	cache_hit_rate
cached	1.349	1217571	0.8302	400323.9	0.9354
naive	1.145	1033837	0.9336	60392.2	N/A

Results and Discussion

Headline findings

Overall winner by mean MB/s: tiktoken at 4.780 MB/s and 0.2195 ms average latency.

Fastest code-domain tokenizer: tiktoken at 5.035 MB/s.

Cached versus naive Python baseline: 1.145 to 1.349 MB/s, or 1.18x speedup, with peak memory rising from 60392.2 to 400323.9 bytes.

Cold start ranking by mean latency favored naive (0.1142 ms) and tiktoken (0.1142 ms), while hf was slowest at 0.6362 ms.

Batch throughput favored the cached baseline at 13481.1 docs/s, but tiktoken is absent from the batch table because the current adapter intentionally exposes single-document encoding only.

Discussion

The results support H1. Native tokenizer libraries clearly outperformed the Python baselines on MB/s and latency, with tiktoken and SentencePiece consistently ahead of naive Python code.

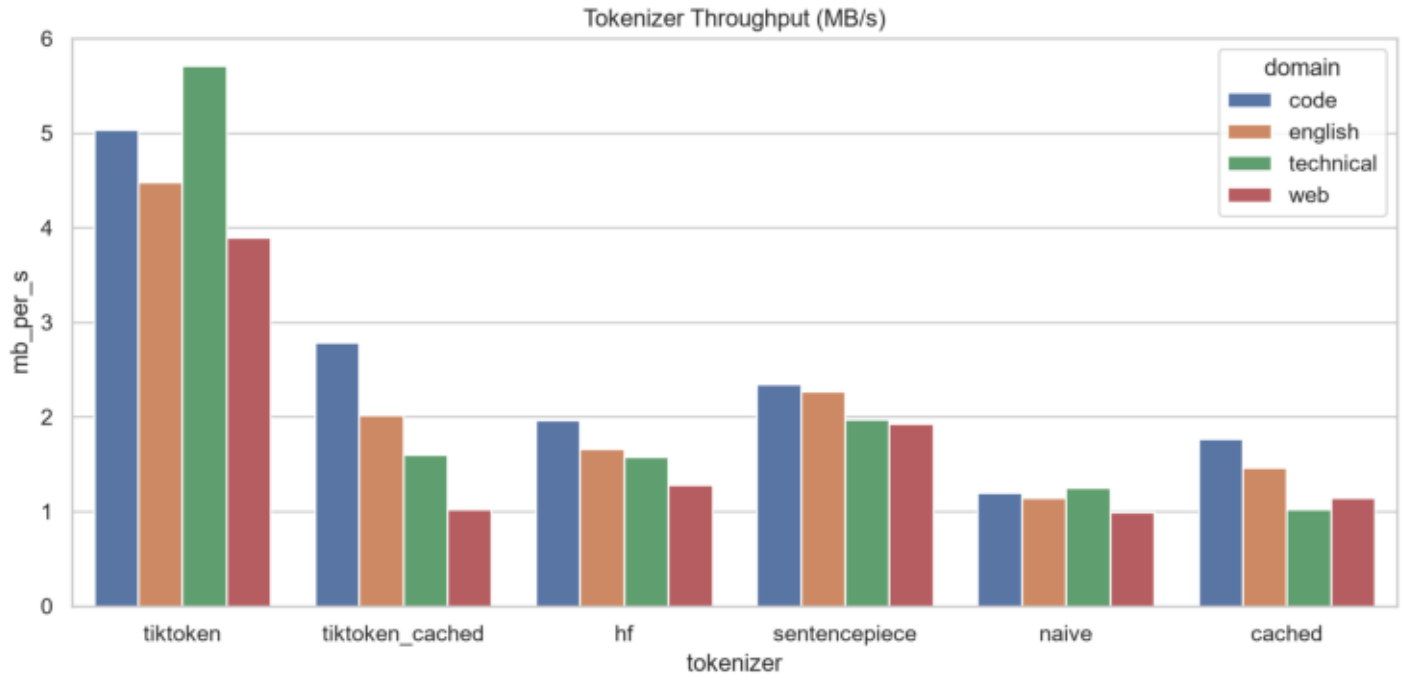
The results support H2. Ranking changed by domain: tiktoken led English, web, and technical text, while SentencePiece led code. That matters because tokenizer selection for RAG preprocessing, web corpora, and code corpora should not rely on one aggregate benchmark number.

The results support H3 with caveats. Caching materially improved the Python baseline, but it increased memory and still depended on high substring reuse. Even after diversifying the synthetic corpus, the cache hit rate remained high, so the next step is to rerun on real corpora.

The results partially support H4. Batch throughput was much higher for adapters with encode_batch support, but batch results are not comparable for tiktoken until a batch-capable adapter path is added.

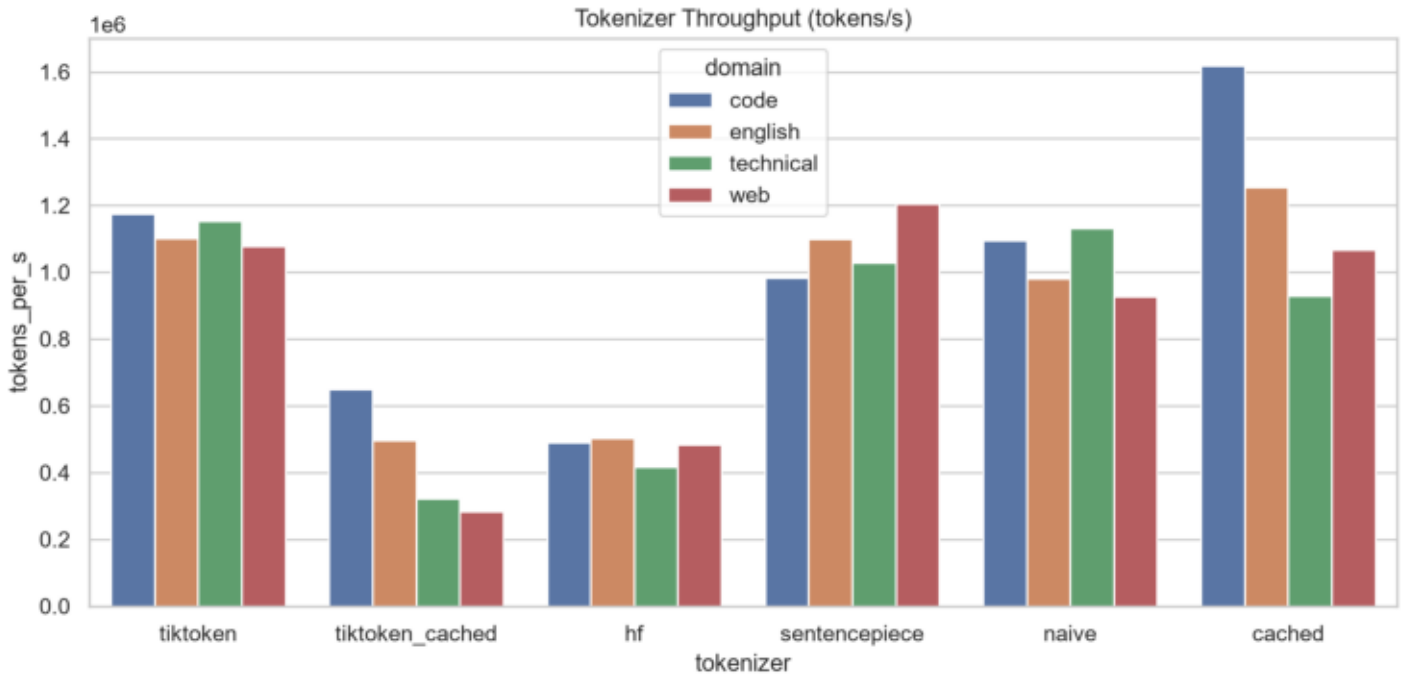
H5 remains unresolved empirically. The benchmark infrastructure is ready for exactness validation, but the project still needs a truly tiktoken-compatible optimized tokenizer before any claim about exact accelerated OpenAI-compatible BPE can be defended.

Throughput Mb S



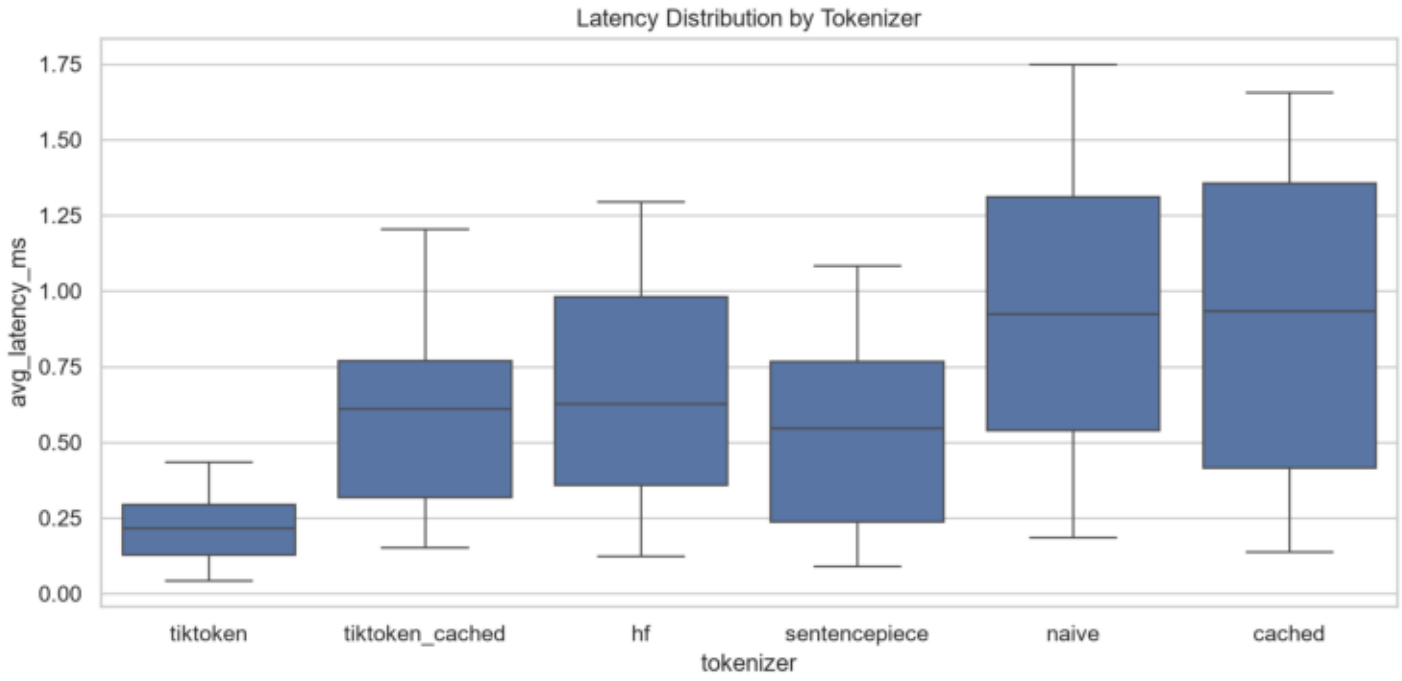
Experiment 1: Tokenizer throughput comparison in MB/s.

Throughput Tokens S



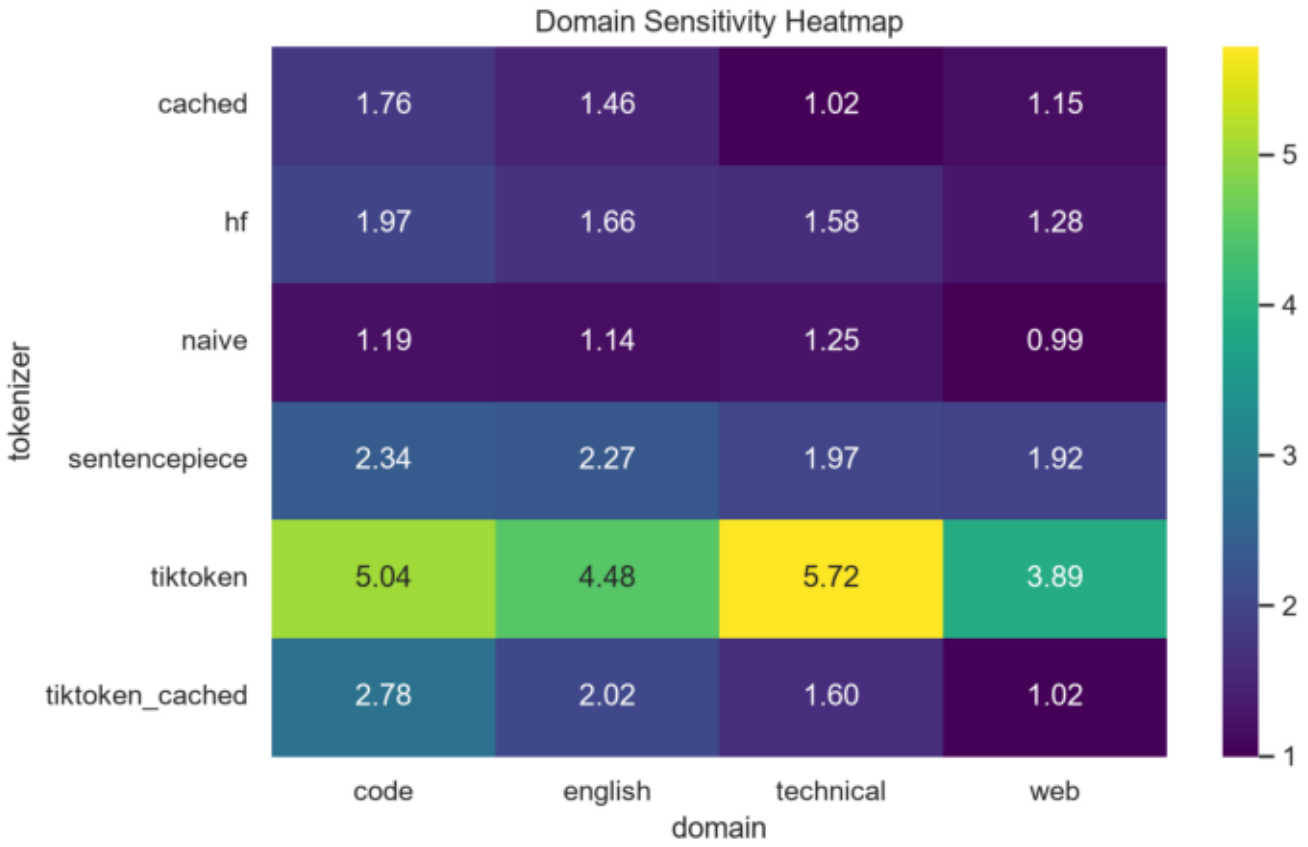
Experiment 1: Tokenizer throughput comparison in tokens/s.

Latency Distribution



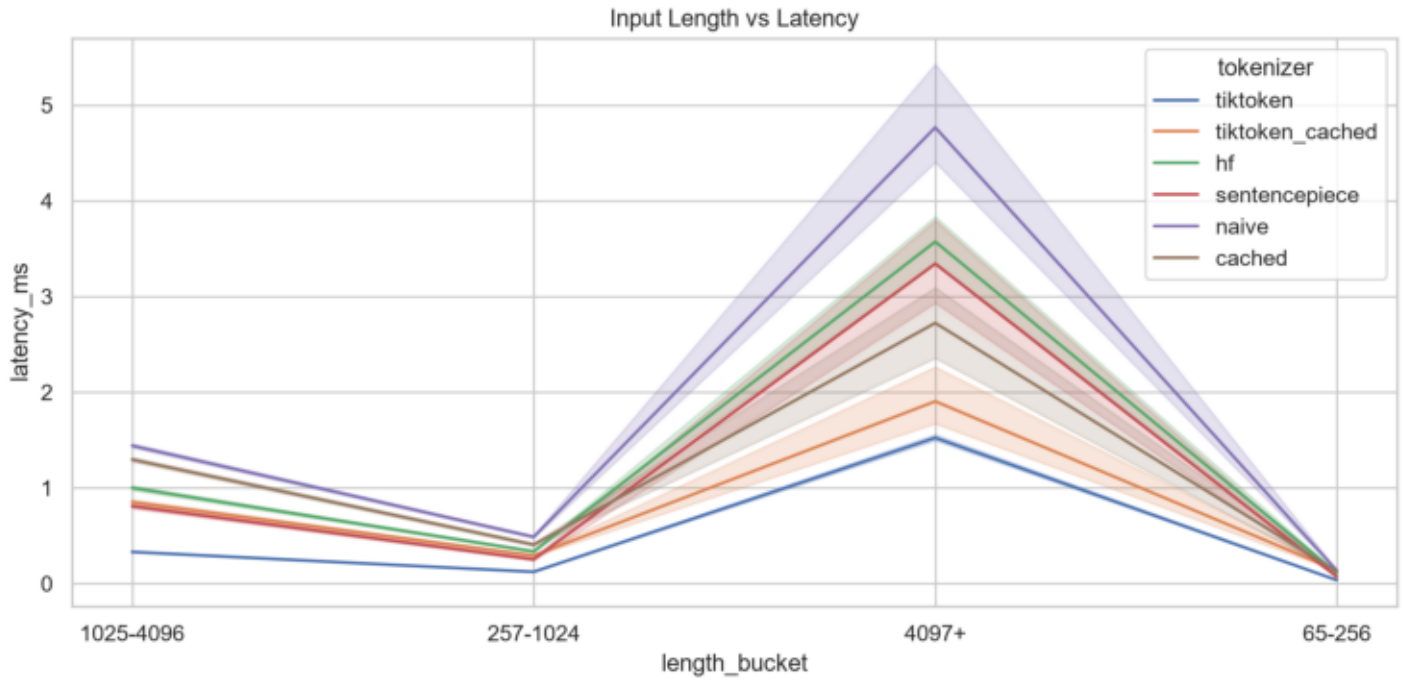
Latency distribution by tokenizer across benchmark trials.

Domain Heatmap



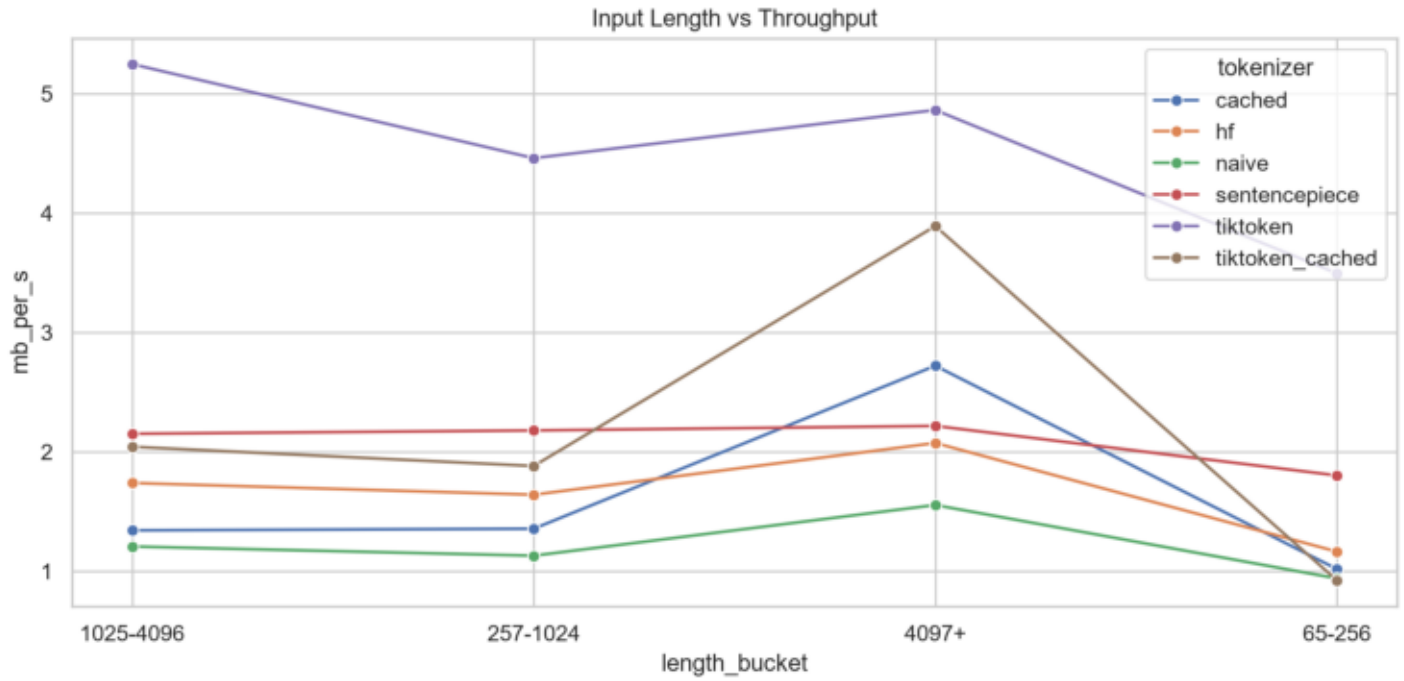
Experiment 2: domain sensitivity heatmap for throughput.

Length Vs Latency



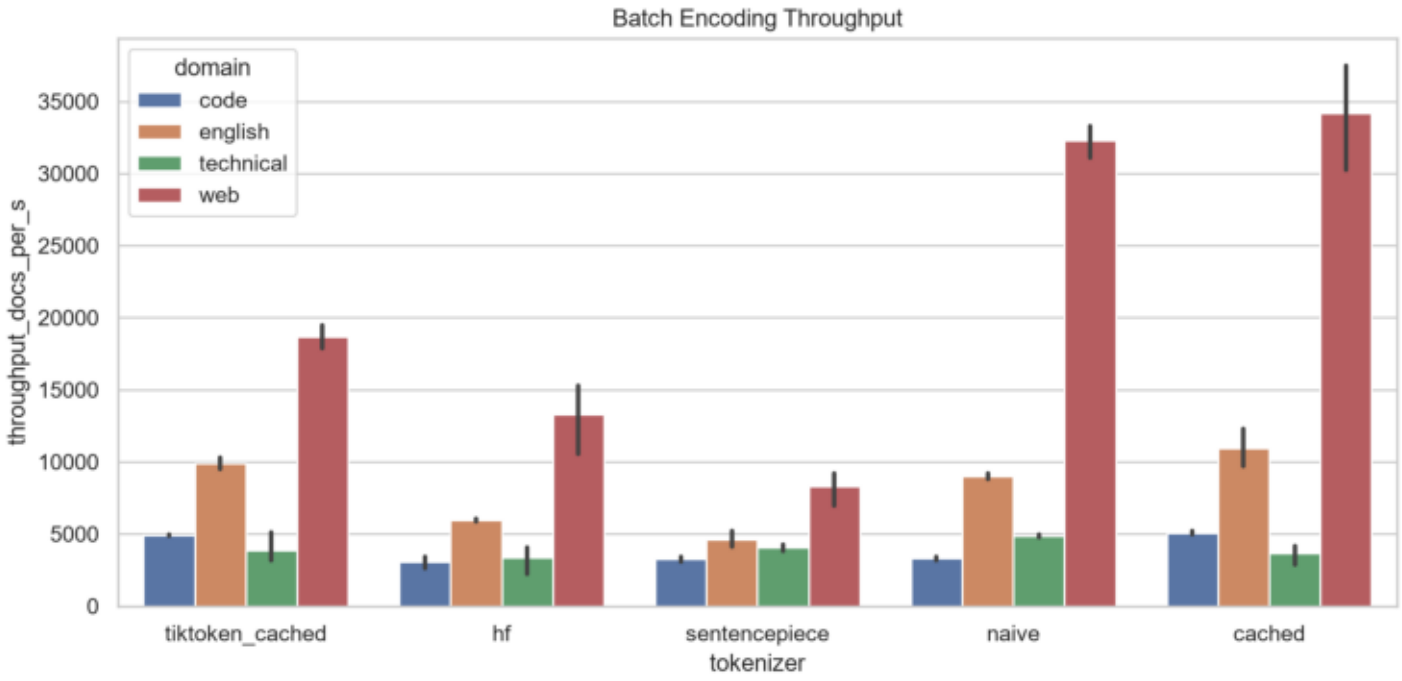
Experiment 3: input length versus latency.

Length Vs Throughput



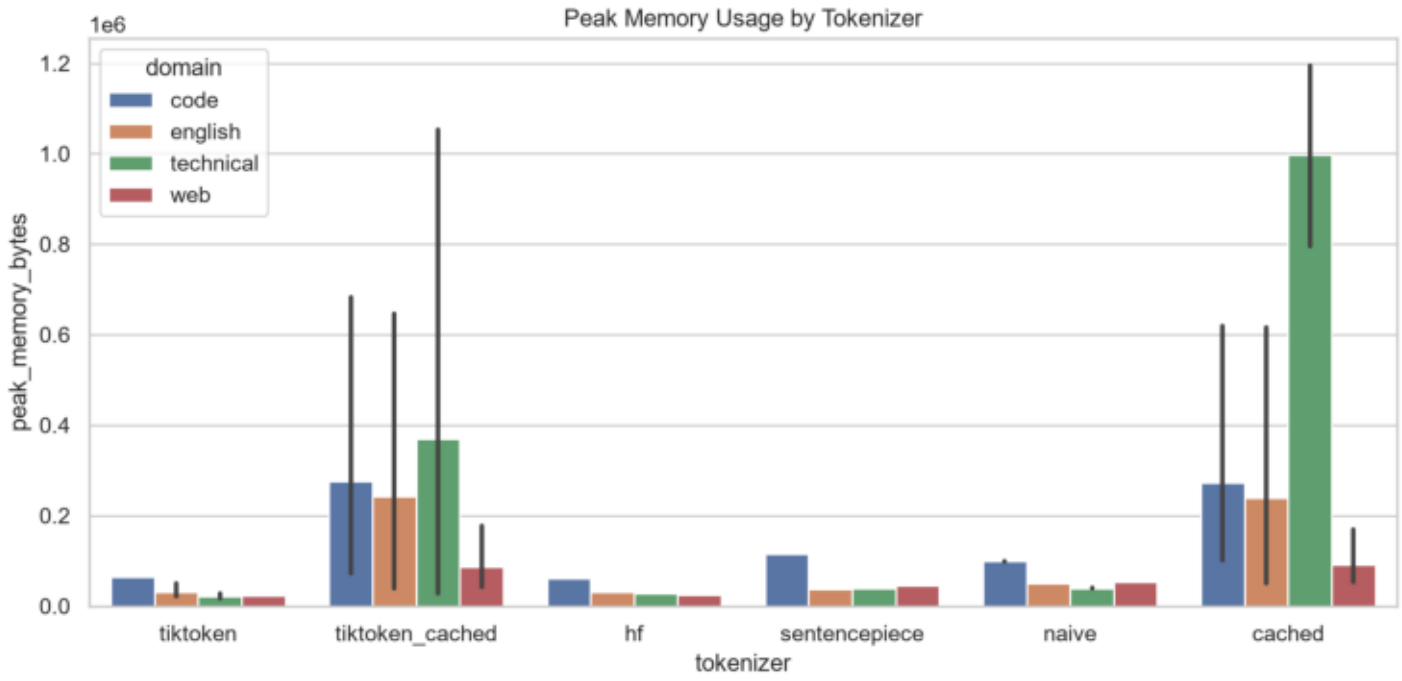
Experiment 3: input length versus throughput.

Batch Throughput



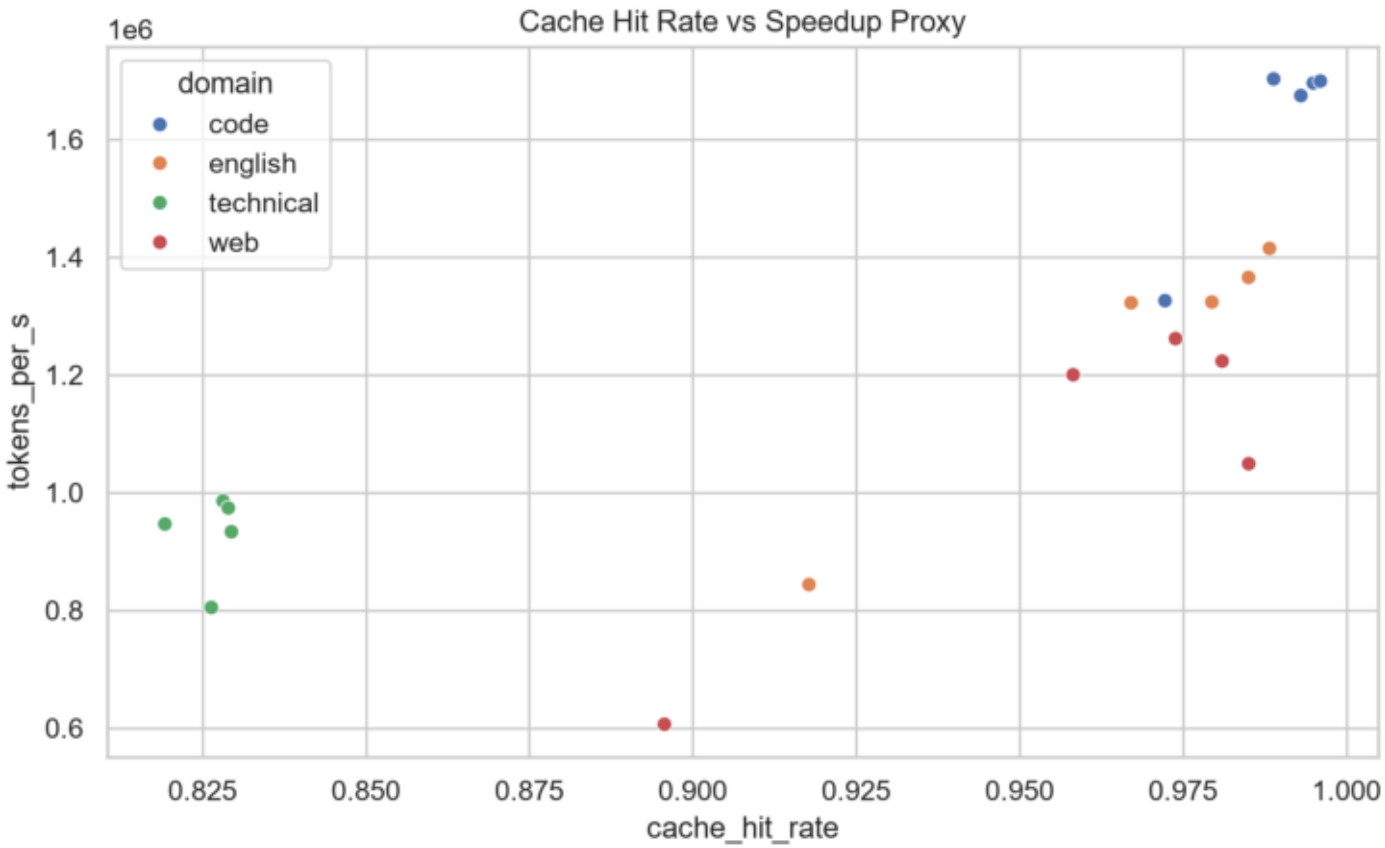
Experiment 4: batch encoding throughput.

Memory Usage



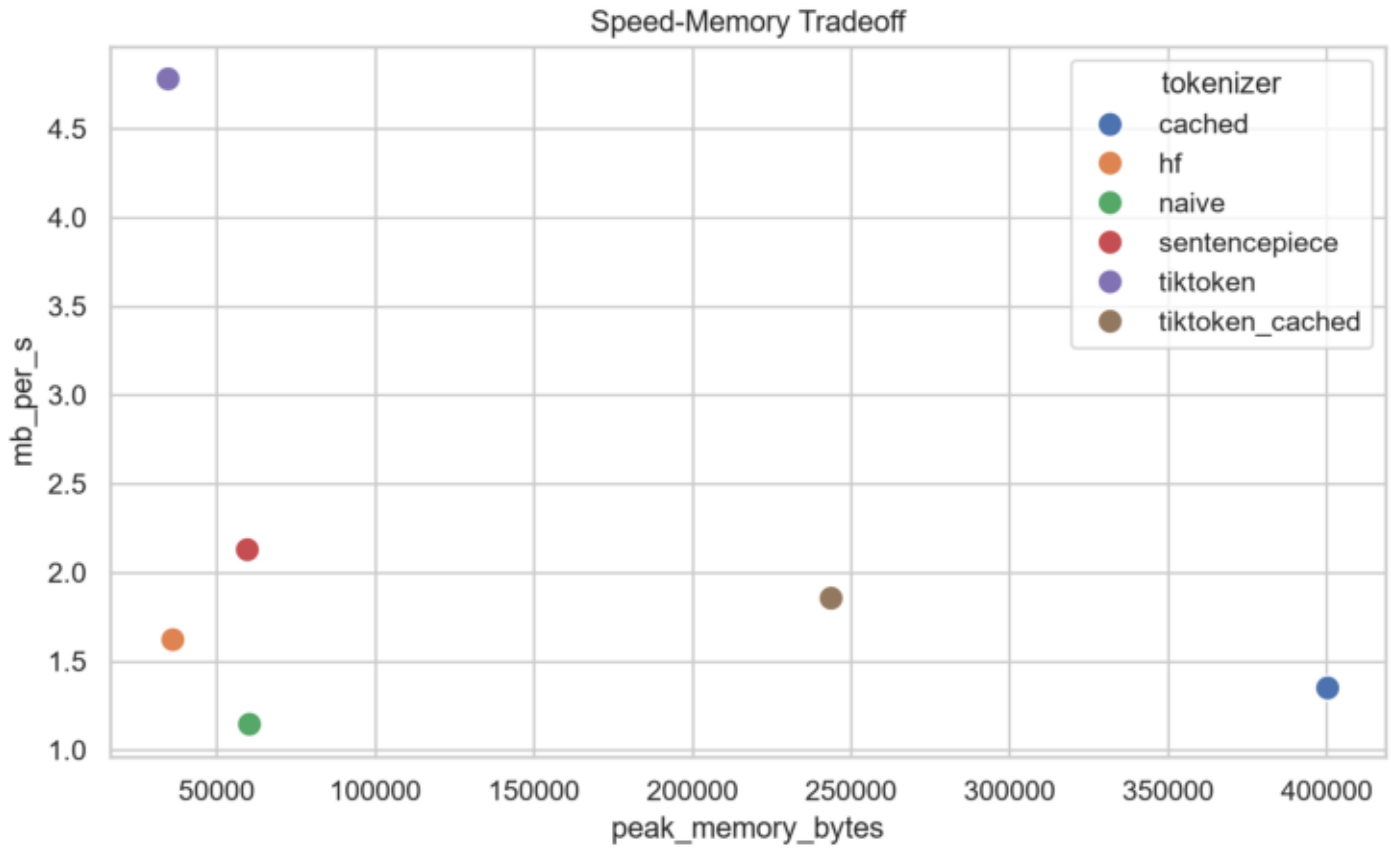
Memory usage by tokenizer and domain.

Cache Hit Rate Vs Speedup



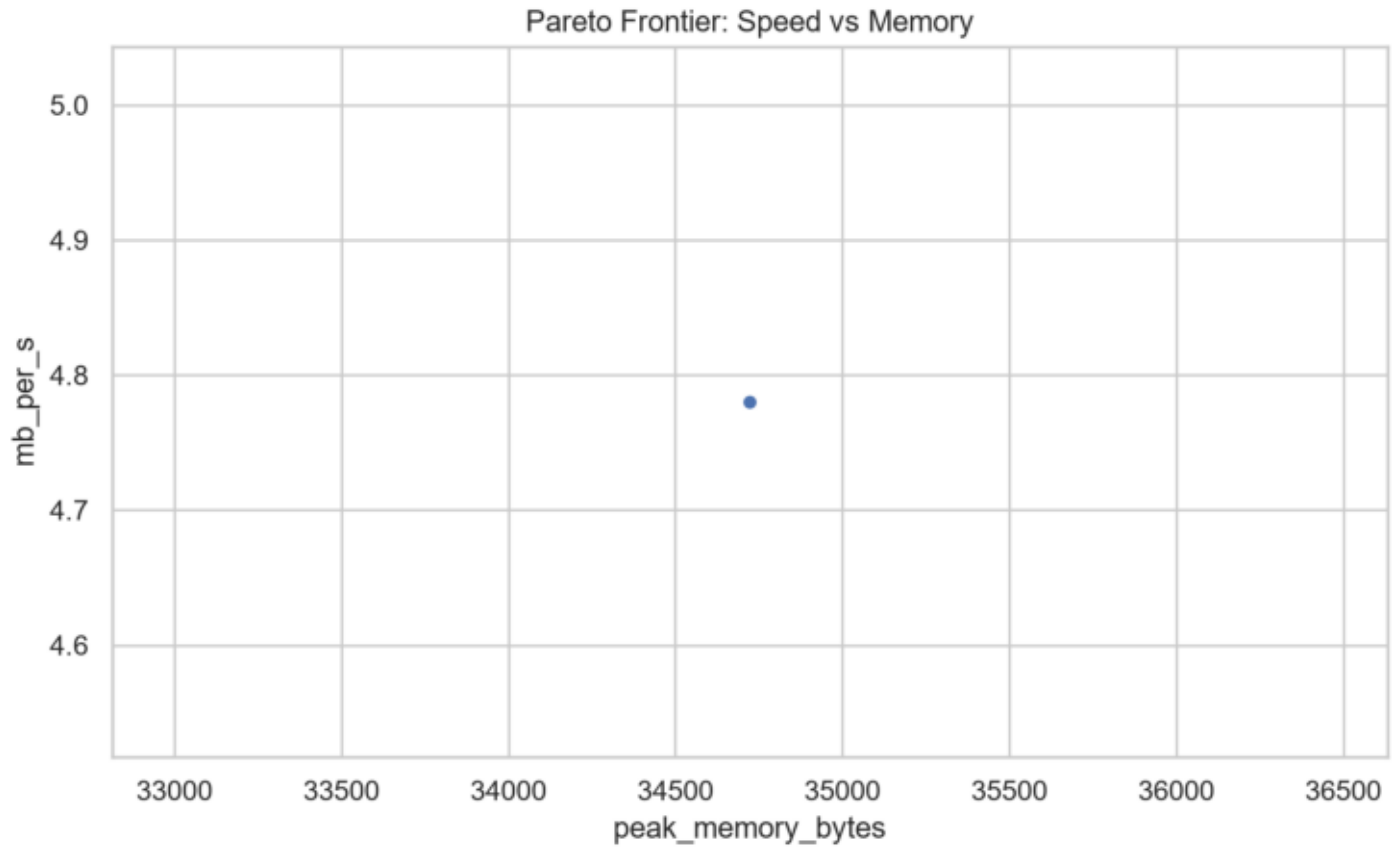
Experiment 6: cache hit rate versus throughput proxy.

Speed Memory Tradeoff



Speed-memory tradeoff across tokenizer implementations.

Pareto Frontier



Pareto frontier of throughput versus memory.